Mohsin Ali

Software Engineer

mohsin.ali.work686@gmail.com
linkedin.com/in/mohsin-ali-63b459293 | mohsinaliwork686-rgb.github.io | github.com/mohsinaliwork686-rgb

Technical Skills

- **Languages**: Python, C#, JavaScript, TypeScript
- **Frameworks**: ASP.NET Core, FastAPI, React, Next.js, Node.js, Azure Functions, Express, Flask, Tailwind CSS
- **Databases**: PostgreSQL, SQL Server, MySQL, MongoDB, Redis
- **Cloud & DevOps**: Microsoft Azure, Azure Durable Functions, Docker, Kubernetes, Helm, GitHub Actions, Bitbucket Pipelines, Prometheus, Git, Linux
- **AI & LLMs**: Claude and OpenAI APIs, retrieval (RAG) evaluation, embeddings, BM25, prompt and context-window tooling

Professional Experience

SoftBuilds

Software Engineer August 2025 – Present

AI Refinery, the company's self-hosted LLM platform July 2026 – Present

Runs open-source LLMs on a customer's own Kubernetes cluster (Helm, Traefik, KServe, Knative, llama.cpp), with a cloud model catalog and subscriptions on Azure (Functions, PostgreSQL, Static Web Apps). My part: subscription enforcement, billing and licensing across both sides, with 232 tests.

- Designed and built subscription enforcement for customers' installations: each one verifies a cloud-signed RS256 entitlement locally, so enforcement needs no internet; an hourly check re-reads the signed token rather than local database values and detects a rolled-back clock; and a locked installation answers 402 on every model endpoint and shows a lock screen, with a grace-period banner before the cutoff (Python, FastAPI, React).
- Made it hold up against tampering: access gated on plan status as well as end date; the grace cutoff signed into the token so it can't be extended locally; offline unlock with a pasted signed token for air-gapped sites; clock-rollback recovery without internet; and subscriptions and API tokens scoped per environment, so a sandbox payment can't unlock production.
- Moved billing from Stripe to Paddle, then added Stripe back behind a provider switch: signed webhooks, each event processed once, monthly and yearly plans, and a one-time trial.
- Designed "one plan, one system" licensing: a permanent ID per installation, claims serialized with PostgreSQL advisory locks and a partial unique index, self-service and admin release, and offline tokens bound to one machine.
- Built negotiated custom plans end to end, from an admin offer-to-checkout flow in the cloud to per-feature and model-size limits enforced on the installation, which restores values tampered with in its local database.
- Found and closed a gap in the ingress authentication path that let locked installations keep serving models, with 19 new tests and an end-to-end check through Traefik.
- Tested changes end to end on local k3d clusters through Traefik, deployed to Azure by hand while CI was down, traced a model-sync failure across both services to a rejected API key, and traced a doubling of the Azure bill to model downloads streaming through one Function.

Healthcare order fulfillment August 2025 – Present

Two separate live systems for two US healthcare clients, fulfilled through LabCorp: **Ditek** (B2B site-based testing) and **Medzah** (direct-to-consumer AtHome testing). Owned the shared Azure services behind both.

- Architected and delivered the ASP.NET Core AtHome platform with ShopPro and LabCorp routing integrations, enabling end-to-end order processing and kit fulfillment at scale.
- Enforced HIPAA-aligned data security by implementing Fernet encryption at rest and building secure PHI handling layers across the order processing pipeline.
- Engineered a blob journaling and event-replay system on Azure Storage, enabling full auditability, operation traceability, and point-in-time recovery for critical order workflows.
- Built a reusable .NET utility microservice consolidating Azure Blob Storage access, SAS URL generation, PDF merging, compression, and document transformation into a single shared service layer.

- Developed automated document-generation pipelines for HTML-to-PDF and image-to-PDF conversion, streamlining requisition and fulfillment documentation across healthcare workflows.
- Built Python Azure Durable Functions orchestrating the full order intake pipeline — from CSV ingestion and requisition generation to shipping label creation, return processing, and refund handling.
- Integrated Endicia and FedEx shipping APIs to fully automate label generation, shipment tracking, and logistics handoffs within the fulfillment pipeline.
- Designed a TRF PDF overlay engine that dynamically stamped order data onto requisition templates, eliminating manual document preparation and accelerating kit dispatch.
- Implemented Node.js Durable Function orchestrators with scheduled triggers to drive multi-stage asset generation pipelines reliably and at scale.
- Owned CI/CD configuration via Bitbucket Pipelines, managing multi-environment deployments across Development, UAT, and Production to ensure consistent, automated releases.

Flippin Technologies

Web Developer January 2025 – July 2025

- Built a full-stack MERN e-commerce platform with end-to-end purchase flows, product management, and Braintree payments.
- Designed and implemented a responsive admin dashboard in Next.js and TypeScript, leveraging Tailwind CSS, ShadCN/UI components, and interactive data visualizations.
- Built a full-stack gamified learning platform with progressive challenges, JWT authentication and role-based access, progress kept in MongoDB, and a suite of mini-games: a Sketchbook, a Spelling Bee on the Speech Synthesis API, Shape Matching with React DnD, and a Reflex Game on Kaplay.

Open Source

- **StackExchange.Redis** **(contributor, merged)** *(C#)*: fixed batched commands missing from the client's profiler, a bug open since 2022, by sharing one profiling step between single and batched commands; regression test included; merged the same day and shipped in release 3.4.29.
- **Swashbuckle.AspNetCore** **(contributor, 3 merged)** *(C#)*: fixed three bugs open since 2022: crashes with `UseAllOfForInheritance` on open generic subclasses and non-public Newtonsoft members (#4244, narrowed after review), and `IReadOnlySet<T>` missing `uniqueItems` (#4246); tested on .NET 8, 9 and 10; shipped in v11.0.0.
- **graphql-code-generator** **(contributor, 2 merged)** *(TypeScript)*: fixed two bugs open since 2024, each checked with `tsc --strict`: generated code that did not compile in string mode with no documents (shipped in `client-preset` 7.0.0), and enums mapped to objects typed as `null` in resolvers (#11116, shipped in 7.0.1).
- **Authlib** **(contributor, 2 merged)** *(Python, OAuth)*: fixed two token-renewal bugs in the client shared by the Requests and HTTPX integrations: expired `client_credentials` tokens never renewed (open since 2024), and renewals losing `access_token_params` (#1026).
- **redis-py** **(contributor, merged)**: documented connection pool limits and `BlockingConnectionPool` after redis-py 8.0 lowered the default to 100 connections; two review rounds with a maintainer.
- **llm-router** *(Python, FastAPI, Redis)*: OpenAI-compatible LLM router with policy routing, fallback, and a daily spending cap that holds under concurrent load through atomic Redis reservations. 236 tests.
- **helmdrift** *(Python, Kubernetes, Helm)*: read-only CLI that finds hand-made changes in a cluster against its Helm chart and names the tool behind each, from `managedFields`. CI runs it on a real kind cluster.
- **semcache** *(Python, Redis)*: semantic cache for LLM calls, in memory or on Redis 8 vector sets, with a benchmark of 250 labelled questions that shows where it starts giving wrong answers. 110 tests; CI on a real Redis and the real embedding model.
- **ragsweep** *(Python)*: sweeps chunking, embeddings and BM25 against a labelled question set to find the retrieval settings that work; 54 runs in 1m 41s, 1.3s when re-run.
- **evalboard** *(TypeScript, Next.js, Recharts)*: in-browser comparison of two eval runs that puts regressions first, where an average hides them; reads ragsweep's results file, and a browser test proves loaded files are never sent anywhere.
- **contextlab** *(TypeScript, Next.js)*: in-browser planner for an LLM context window: what fits, what it costs, and where a long request gets cut.
- **promptlint** *(Python)*: linter with autofix for LLM prompt templates: broken formatting, missing variables, leaked keys.

Education

UET Lahore Lahore

Bachelor in Computer Science